

LLD Practice Problems — Set 2 (Singleton · Strategy · Decorator)

Three class-design problems. Don't start from the pattern — start from the requirements, let the pattern emerge, then name it.

Ground rules: records, immutability, no nulls, dependencies through constructors.

Problem 1 — Application Config Store (*Singleton*)

The story

Every part of your app needs configuration: DB URL, feature flags, API keys. Config is loaded from a file — an **expensive** operation (~200ms). It must load **at most once**, and only **when first needed**.

Requirements

1. `AppConfig.getInstance()` returns the same instance everywhere; the file loads exactly once, on first use (lazy).
2. At startup, many threads may call `getInstance()` at the same time — the file must still load exactly once.
3. `get(String key)` — the key may be absent. Never return null; choose the right return type.
4. `getInt(String key, int defaultValue)` and `getBoolean(String key, boolean defaultValue)`.
5. Values never change after load — the config object is immutable.

Questions to answer

1. Show why the naive lazy version (`if (INSTANCE == null) INSTANCE = new AppConfig();`) breaks under two simultaneous threads.
 2. Implement it with the **holder idiom**, then again as an **enum**. One or two lines each — which do you prefer and why?
 3. Your tests need a different config (test DB URL). Is `getInstance()` hardcoded inside classes a problem now? What's the alternative that keeps "one instance" — and where does that one instance get created?
-

Problem 2 — Cab Fare Calculator (*Strategy*)

The story

Finance changes the fare formula every week:

- **Week 1:** `base + perKm × distance`
- **Week 2:** night rides (10pm–6am) cost 1.25×
- **Week 3:** surge — multiply by current demand factor
- **Week 4:** flat ₹49 for rides under 2 km

The trip pipeline — validate, compute distance, apply fare, create invoice — must **never change** when the formula does.

Requirements

1. `record Trip(double distanceKm, LocalTime startTime, double demandFactor) { }`
2. A fare policy is a function: `Trip → int` (fare in rupees). Use `Function<Trip, Integer>` or your own `@FunctionalInterface FarePolicy`.
3. Implement all four weekly policies as lambdas or small static methods.
4. `FareService` receives the policy through its constructor. Switching the company's fare scheme = changing **one line** in `main`.
5. Unit-test the week-2 policy with an 11pm trip — no service needed, just the function.

Questions to answer

1. Why is a lambda enough here? (What do all four policies have in common — state? dependencies?)
2. Week 2 is really "week 1's result $\times 1.25$ ". Can you build it by **reusing** the week-1 function instead of copy-pasting the formula?

Problem 3 — Notification Text Pipeline (*Decorator*)

The story

Before a notification goes out, its text passes through transformations — and each channel needs a different sequence:

Transformation	What it does
<code>trim</code>	remove leading/trailing whitespace
<code>maskProfanity</code>	replace banned words with <code>***</code>
<code>truncate(160)</code>	hard limit for SMS
<code>appendSignature</code>	adds "— Team MiniZomato"
<code>capitalizeFirst</code>	uppercase the first letter

- **SMS:** `trim` → `maskProfanity` → `truncate(160)`
- **EMAIL:** `trim` → `maskProfanity` → `capitalizeFirst` → `appendSignature`
- **PUSH:** `trim` → `truncate(120)`

Marketing adds new transformations every month and reshuffles the order without warning.

Requirements

1. Every transformation is a `Function<String, String>`. `truncate` is parameterized — write it as a method that **returns** a function: `static Function<String, String> truncate(int max)`.
2. Build each channel's pipeline by composing with `andThen` (or a `reduce` with `Function.identity()` over a list of functions).
3. Adding a new transformation or reordering a channel must touch **one place only** — no transformation may know about any other, and no `if (channel == SMS)` anywhere.
4. All transformations must be **pure functions**.

Questions to answer

1. Prove order matters: show that `truncate(160)` then `appendSignature` can produce a message longer than 160. Which order is correct for SMS?
 2. Compare your pipeline with the classic decorator (`(new DataInputStream(new BufferedInputStream(new FileInputStream(...))))`). What's the same? What did `andThen` save you from writing?
 3. Why would one impure transformation (say, `maskProfanity` counting hits in a static variable) quietly break this design?
-

Takeaway

All three problems share one shape: **a stable core plus interchangeable pieces supplied from outside** — one instance wired once (config), one decision injected as a function (fare), one chain composed from small pure steps (text). If your solution has that shape, you've used the patterns correctly — whether or not you said their names.